Scaling and Accuracy with `pseudopeople`

This document benchmarks `irelink` on five `pseudopeople` scenarios using the parquet files in the sibling `pseudopeople-ri` repository. Every scenario reads the parquet files directly through DuckDB, so the full source tables stay out of R memory.

The same comparison specification and the same three blocking rules are used across all five tests:

1. `first_name + last_name`
2. `last_name + date_of_birth`
3. `zipcode + date_of_birth`

```
devtools::load_all()
```

```
ℹ Loading irelink
```

```
library(dplyr)
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(knitr)
library(tictoc)

pseudopeople_dir <- '../../../pseudopeople-ri'

zero_path <- paste0(pseudopeople_dir, '/psp_census_2020_zero_noise.parquet')
default_path <- paste0(
  pseudopeople_dir,
  '/psp_census_2020_default_noise.parquet'
)
high_path <- paste0(pseudopeople_dir, '/psp_census_2020_high_noise.parquet')
```

```
common_spec <- il_spec() |>
  il_compare(first_name, cl_name()) |>
  il_compare(last_name, cl_name()) |>
  il_compare(date_of_birth, cl_damerau_levenshtein(1, 2)) |>
  il_compare(street_number, cl_damerau_levenshtein(1)) |>
  il_compare(city, cl_exact()) |>
  il_compare(zipcode, cl_damerau_levenshtein(1)) |>
  il_block_on(first_name, last_name) |>
  il_block_on(last_name, date_of_birth) |>
  il_block_on(zipcode, date_of_birth)
```

No noise to zero

```
zero_zero_con <- DBI::dbConnect(duckdb::duckdb())

DBI::dbExecute(
  zero_zero_con,
  paste0(
    'CREATE VIEW zero_zero_left AS ',
    "SELECT * FROM read_parquet('",
    zero_path,
    "')"
  )
)
```

```
[1] 0
```

```
DBI::dbExecute(
  zero_zero_con,
  paste0(
    'CREATE VIEW zero_zero_right AS ',
    "SELECT * FROM read_parquet('",
    zero_path,
    "')"
  )
)
```

```
[1] 0
```

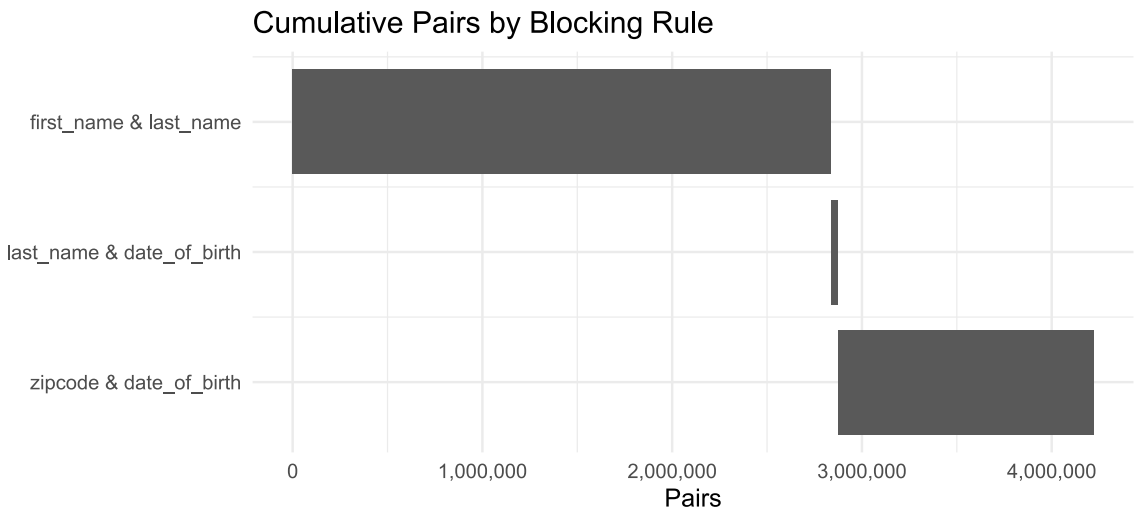
```
zero_zero_left <- tbl(zero_zero_con, 'zero_zero_left')
zero_zero_right <- tbl(zero_zero_con, 'zero_zero_right')
```

Blocking

```
tic.clearlog()
tic('zero_zero_blocking', quiet = TRUE)
zero_zero_blocking <- il_count_pairs(
  zero_zero_left,
  zero_zero_right,
  block_on(first_name, last_name),
  block_on(last_name, date_of_birth),
  block_on(zipcode, date_of_birth),
  con = zero_zero_con,
  link_type = 'link'
)
zero_zero_blocking_timing <- toc(log = TRUE, quiet = TRUE)
zero_zero_blocking_time <- as.numeric(
  zero_zero_blocking_timing$toc - zero_zero_blocking_timing$tic
)
zero_zero_candidate_pairs <- max(zero_zero_blocking$cumulative_pairs)
zero_zero_cartesian_share <- max(zero_zero_blocking$pct_of_cartesian)
```

This is the clean ceiling case: the same zero-noise census file appears on both sides, so the benchmark should be close to a keyed merge. The blocking rules reduce the search to 4,219,926 candidate pairs, which is 0.04% of the full cross-product.

```
autoplot(zero_zero_blocking)
```



Model fit

```
tic.clearlog()
tic('zero_zero_fit', quiet = TRUE)
```

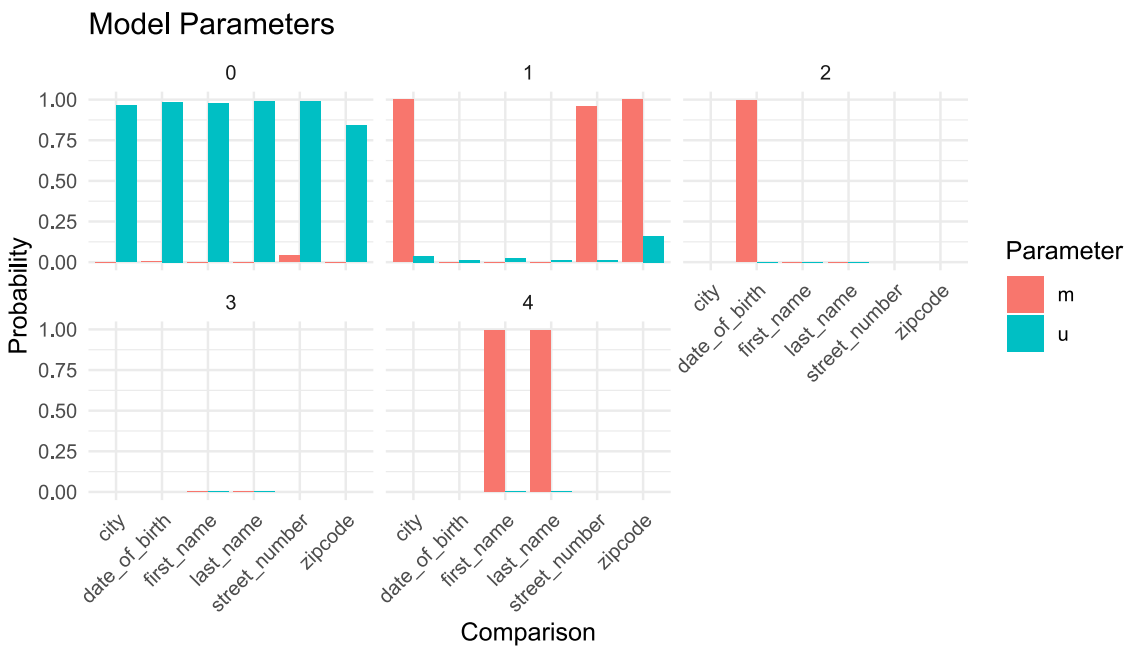
```
zero_zero_model <- il_model(
  zero_zero_left,
  zero_zero_right,
  spec = common_spec,
  con = zero_zero_con,
  link_type = 'link'
) |>
  il_estimate_prior(
    block_on(first_name, last_name, date_of_birth),
    recall = 0.8
  ) |>
  il_estimate_u(max_pairs = 5e5) |>
  il_estimate_em(
    blocking = block_on(first_name, last_name),
    max_iterations = 8L
  ) |>
  il_estimate_em(
    blocking = block_on(zipcode, date_of_birth),
    max_iterations = 8L
  )
```

```
EM trained: date_of_birth, street_number, city, and zipcode | skipped (blocked
on): first_name and last_name
EM trained: first_name, last_name, street_number, and city | skipped (blocked
on): date_of_birth and zipcode
```

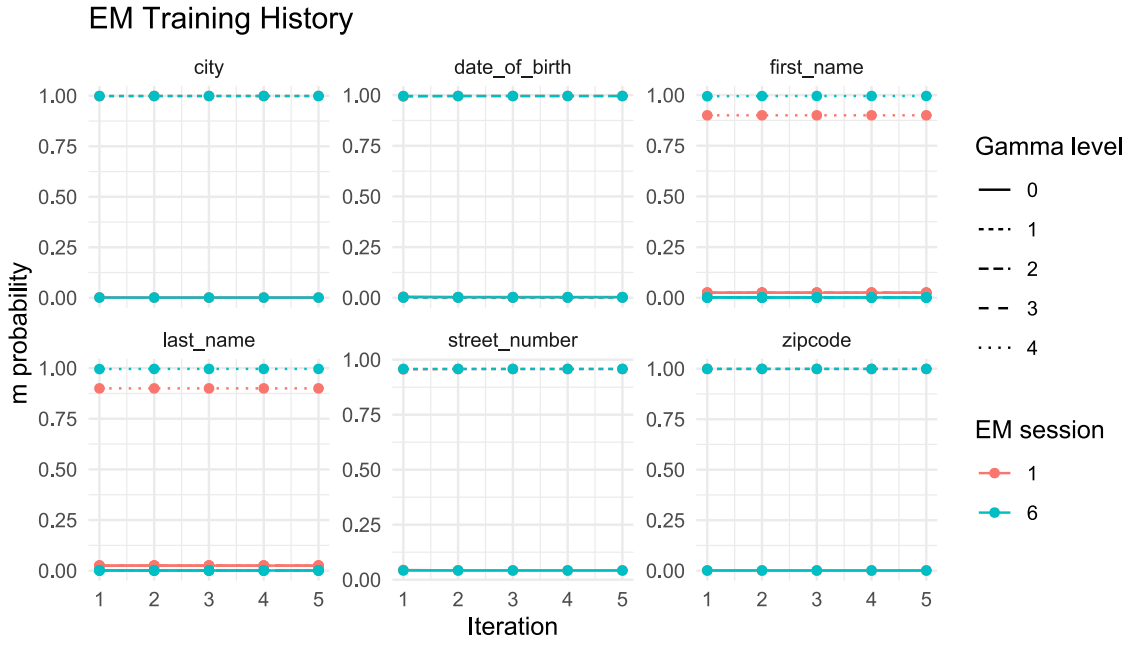
```
zero_zero_fit_timing <- toc(log = TRUE, quiet = TRUE)
zero_zero_fit_time <- as.numeric(
  zero_zero_fit_timing$toc - zero_zero_fit_timing$tic
)
```

The model fit includes prior estimation, u-estimation, and two EM sessions. This step takes 8.4 seconds in the clean linkage ceiling case.

```
autoplot(zero_zero_model, type = 'parameters')
```




```
il_training_history(zero_zero_model) |>
  autoplot()
```

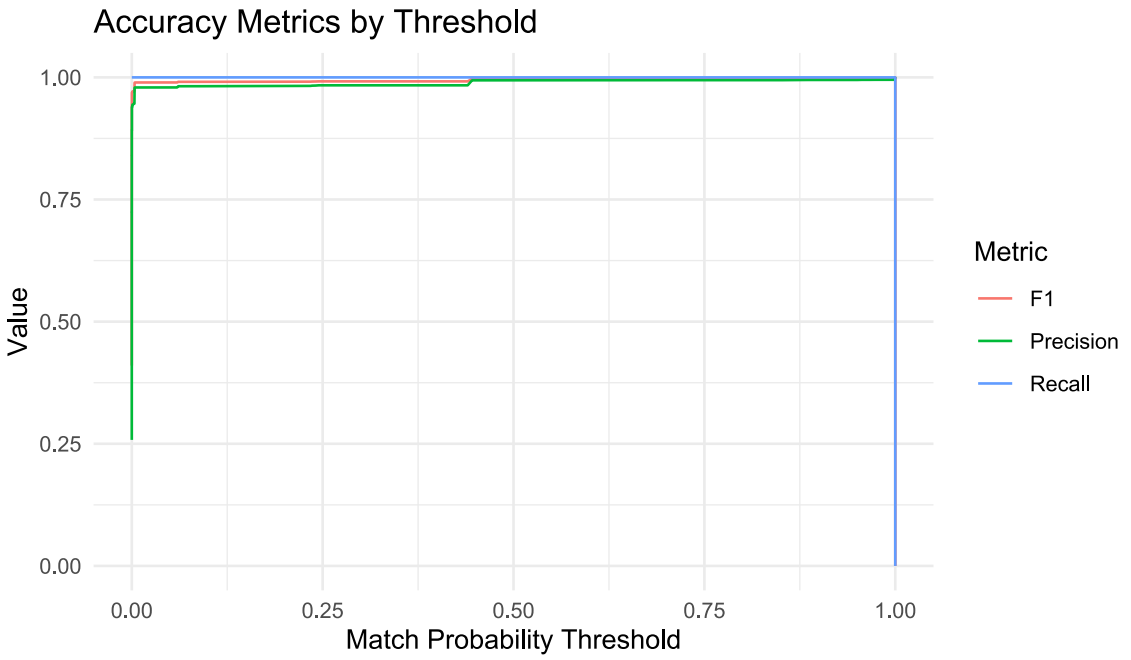


Accuracy

```
tic.clearlog()
tic('zero_zero_accuracy', quiet = TRUE)
zero_zero_accuracy <- il_accuracy(
  zero_zero_model,
  labels_col = 'simulant_id'
)
zero_zero_accuracy_timing <- toc(log = TRUE, quiet = TRUE)
zero_zero_accuracy_time <- as.numeric(
  zero_zero_accuracy_timing$toc - zero_zero_accuracy_timing$tic
)
zero_zero_best <- zero_zero_accuracy |>
  slice_max(f1, n = 1, with_ties = FALSE)
```

At the best threshold (1), precision is 100%, recall is 100%, and F1 is 1. Accuracy evaluation takes 32.8 seconds.

```
autoplot(zero_zero_accuracy)
```



Prediction

```
tic.clearlog()
tic('zero_zero_prediction', quiet = TRUE)
zero_zero_pairs <- predict(
  zero_zero_model,
```

```
  threshold = zero_zero_best$threshold
)
zero_zero_prediction_timing <- toc(log = TRUE, quiet = TRUE)
zero_zero_prediction_time <- as.numeric(
  zero_zero_prediction_timing$toc - zero_zero_prediction_timing$tic
)
zero_zero_predicted_links <- nrow(zero_zero_pairs)

tic.clearlog()
tic('zero_zero_prediction_correctness', quiet = TRUE)
zero_zero_confusion <- il_confusion_matrix(
  zero_zero_model,
  threshold = zero_zero_best$threshold,
  labels_col = 'simulant_id'
)
zero_zero_prediction_correctness_timing <- toc(log = TRUE, quiet = TRUE)
zero_zero_prediction_correctness_time <- as.numeric(
  zero_zero_prediction_correctness_timing$toc -
    zero_zero_prediction_correctness_timing$tic
)
```

At the best F1 threshold, `predict()` returns 1,042,965 links in 2.8 seconds. Thresholded prediction correctness is precision 100%, recall 100%, and F1 1.

Timing

```
zero_zero_timings <- tibble::tibble(
  stage = c(
    'Blocking analysis',
    'Model fitting',
    'Accuracy evaluation',
    'Prediction',
    'Prediction correctness',
    'Total'
  ),
  seconds = c(
    zero_zero_blocking_time,
    zero_zero_fit_time,
    zero_zero_accuracy_time,
    zero_zero_prediction_time,
    zero_zero_prediction_correctness_time,
    zero_zero_blocking_time +
      zero_zero_fit_time +
      zero_zero_accuracy_time +
      zero_zero_prediction_time +
      zero_zero_prediction_correctness_time
  )
)
```

```
zero_zero_timings
```

```
# A tibble: 6 × 2
  stage                  seconds
  <chr>                    <dbl>
1 Blocking analysis         2.72
2 Model fitting             8.45
3 Accuracy evaluation      32.8
4 Prediction                2.83
5 Prediction correctness   18.0
6 Total                    64.9
```

No noise to default

```
zero_default_con <- DBI::dbConnect(duckdb::duckdb())

DBI::dbExecute(
  zero_default_con,
  paste0(
    'CREATE VIEW zero_default_left AS ',
    "SELECT * FROM read_parquet('",
    zero_path,
    "')"
  )
)
```

```
[1] 0
```

```
DBI::dbExecute(
  zero_default_con,
  paste0(
    'CREATE VIEW zero_default_right AS ',
    "SELECT * FROM read_parquet('",
    default_path,
    "')"
  )
)
```

```
[1] 0
```

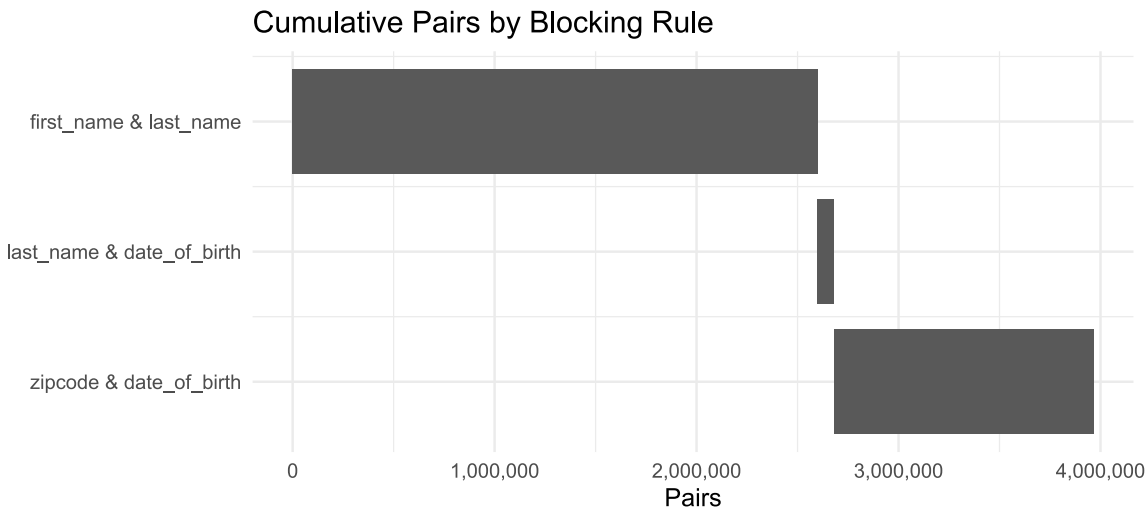
```
zero_default_left <- tbl(zero_default_con, 'zero_default_left')
zero_default_right <- tbl(zero_default_con, 'zero_default_right')
```

Blocking

```
tic.clearlog()
tic('zero_default_blocking', quiet = TRUE)
zero_default_blocking <- il_count_pairs(
  zero_default_left,
  zero_default_right,
  block_on(first_name, last_name),
  block_on(last_name, date_of_birth),
  block_on(zipcode, date_of_birth),
  con = zero_default_con,
  link_type = 'link'
)
zero_default_blocking_timing <- toc(log = TRUE, quiet = TRUE)
zero_default_blocking_time <- as.numeric(
  zero_default_blocking_timing$toc - zero_default_blocking_timing$tic
)
zero_default_candidate_pairs <- max(zero_default_blocking$cumulative_pairs)
zero_default_cartesian_share <- max(zero_default_blocking$pct_of_cartesian)
```

This links the clean census extract to the default-noise administrative extract. The blocking rules keep the candidate space to 3,965,067 pairs, or 0.03% of the full cross-product.

```
autoplot(zero_default_blocking)
```



Model fit

```
tic.clearlog()
tic('zero_default_fit', quiet = TRUE)
zero_default_model <- il_model(
  zero_default_left,
```

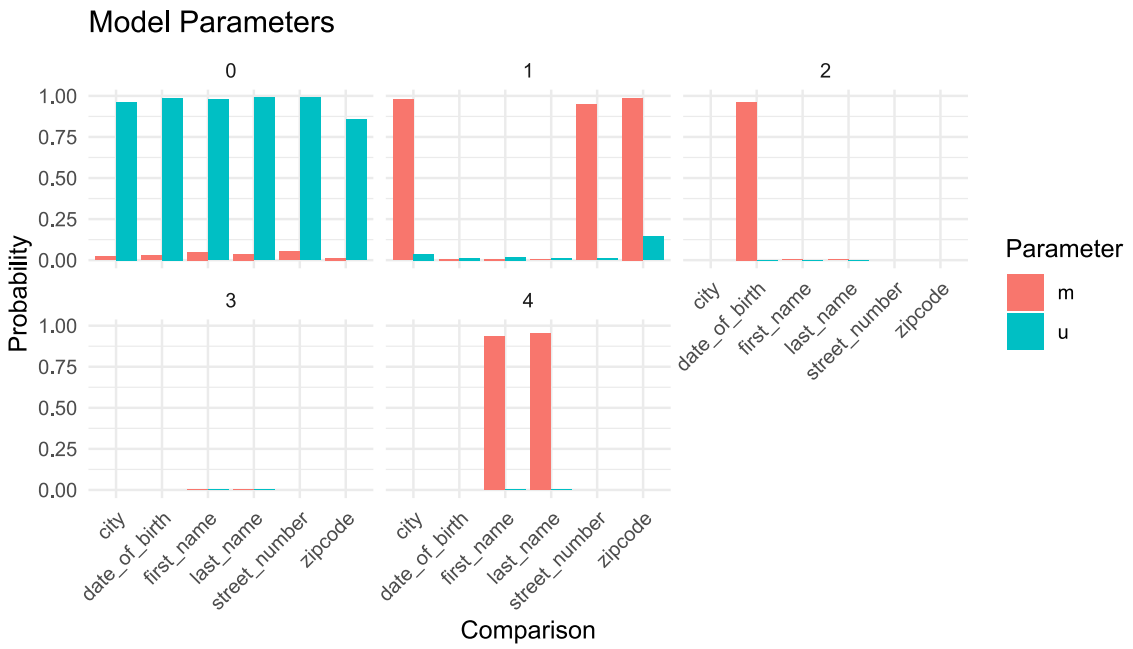
```
  zero_default_right,
  spec = common_spec,
  con = zero_default_con,
  link_type = 'link'
) |>
  il_estimate_prior(
    block_on(first_name, last_name, date_of_birth),
    recall = 0.8
  ) |>
  il_estimate_u(max_pairs = 5e5) |>
  il_estimate_em(
    blocking = block_on(first_name, last_name),
    max_iterations = 8L
  ) |>
  il_estimate_em(
    blocking = block_on(zipcode, date_of_birth),
    max_iterations = 8L
  )
```

```
EM trained: date_of_birth, street_number, city, and zipcode | skipped (blocked
on): first_name and last_name
EM trained: first_name, last_name, street_number, and city | skipped (blocked
on): date_of_birth and zipcode
```

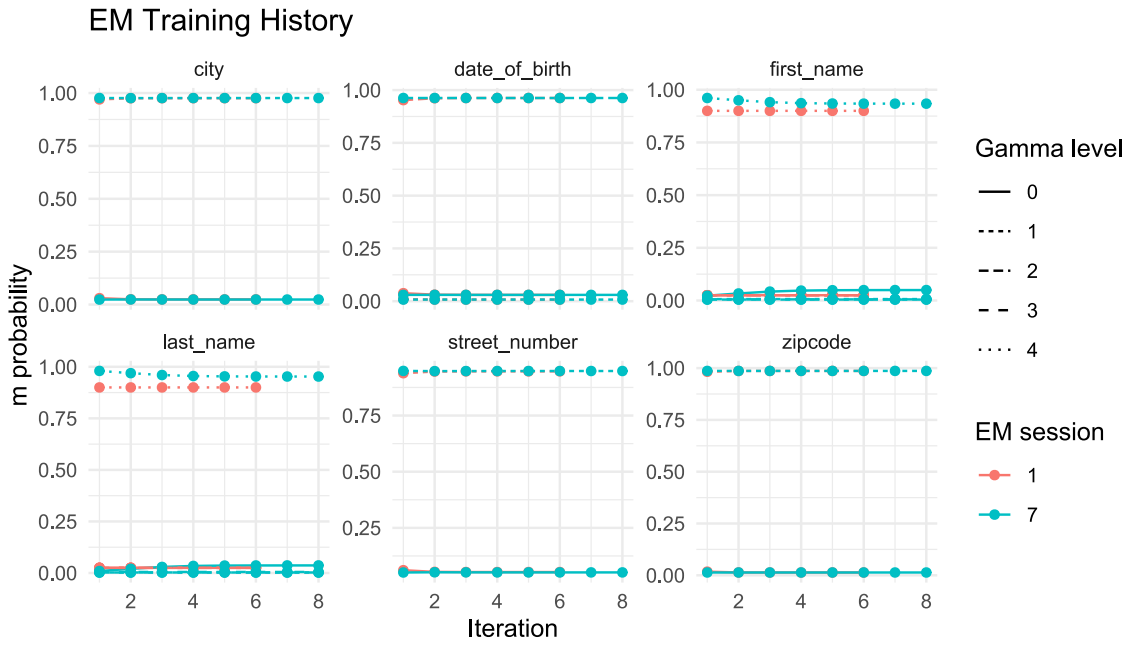
```
zero_default_fit_timing <- toc(log = TRUE, quiet = TRUE)
zero_default_fit_time <- as.numeric(
  zero_default_fit_timing$toc - zero_default_fit_timing$tic
)
```

This fit step takes 7.8 seconds. The same specification is used here, but the default-noise file makes the EM updates materially less clean than in the no-noise-to-zero case.

```
autoplot(zero_default_model, type = 'parameters')
```




```
il_training_history(zero_default_model) |>
  autoplot()
```

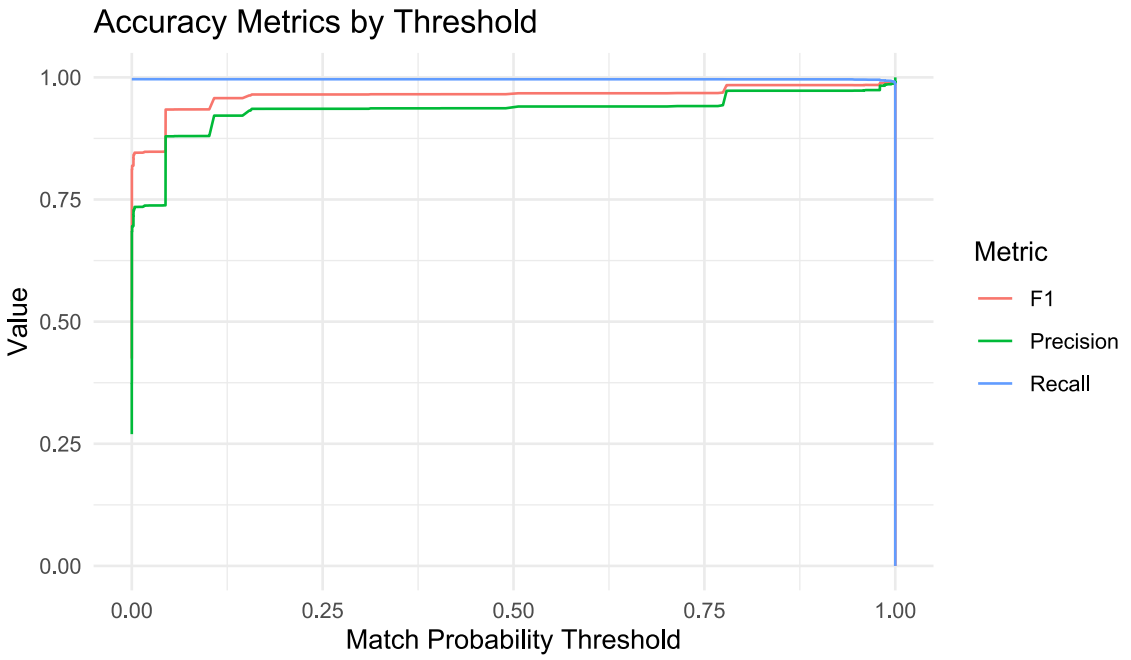


Accuracy

```
tic.clearlog()
tic('zero_default_accuracy', quiet = TRUE)
zero_default_accuracy <- il_accuracy(
  zero_default_model,
  labels_col = 'simulant_id'
)
zero_default_accuracy_timing <- toc(log = TRUE, quiet = TRUE)
zero_default_accuracy_time <- as.numeric(
  zero_default_accuracy_timing$toc - zero_default_accuracy_timing$tic
)
zero_default_best <- zero_default_accuracy |>
  slice_max(f1, n = 1, with_ties = FALSE)
```

At the best threshold (0.995), precision is 98.7%, recall is 99.2%, and F1 is 0.989. Accuracy evaluation takes 60.2 seconds.

```
autoplot(zero_default_accuracy)
```



Prediction

```
tic.clearlog()
tic('zero_default_prediction', quiet = TRUE)
zero_default_pairs <- predict(
  zero_default_model,
```

```
  threshold = zero_default_best$threshold
)
zero_default_prediction_timing <- toc(log = TRUE, quiet = TRUE)
zero_default_prediction_time <- as.numeric(
  zero_default_prediction_timing$toc - zero_default_prediction_timing$tic
)
zero_default_predicted_links <- nrow(zero_default_pairs)

tic.clearlog()
tic('zero_default_prediction_correctness', quiet = TRUE)
zero_default_confusion <- il_confusion_matrix(
  zero_default_model,
  threshold = zero_default_best$threshold,
  labels_col = 'simulant_id'
)
zero_default_prediction_correctness_timing <- toc(log = TRUE, quiet = TRUE)
zero_default_prediction_correctness_time <- as.numeric(
  zero_default_prediction_correctness_timing$toc -
    zero_default_prediction_correctness_timing$tic
)
```

At the best F1 threshold, `predict()` returns 1,079,086 links in 2.8 seconds. Thresholded prediction correctness is precision 98.7%, recall 99.2%, and F1 0.989.

Timing

```
zero_default_timings <- tibble::tibble(
  stage = c(
    'Blocking analysis',
    'Model fitting',
    'Accuracy evaluation',
    'Prediction',
    'Prediction correctness',
    'Total'
  ),
  seconds = c(
    zero_default_blocking_time,
    zero_default_fit_time,
    zero_default_accuracy_time,
    zero_default_prediction_time,
    zero_default_prediction_correctness_time,
    zero_default_blocking_time +
      zero_default_fit_time +
      zero_default_accuracy_time +
      zero_default_prediction_time +
      zero_default_prediction_correctness_time
  )
)
```

```
zero_default_timings
```

```
# A tibble: 6 × 2
  stage                  seconds
  <chr>                    <dbl>
1 Blocking analysis         2.78
2 Model fitting             7.8
3 Accuracy evaluation      60.2
4 Prediction                2.81
5 Prediction correctness   16.2
6 Total                    89.7
```

No noise to high

```
zero_high_con <- DBI::dbConnect(duckdb::duckdb())

DBI::dbExecute(
  zero_high_con,
  paste0(
    'CREATE VIEW zero_high_left AS ',
    "SELECT * FROM read_parquet('",
    zero_path,
    "')"
  )
)
```

```
[1] 0
```

```
DBI::dbExecute(
  zero_high_con,
  paste0(
    'CREATE VIEW zero_high_right AS ',
    "SELECT * FROM read_parquet('",
    high_path,
    "')"
  )
)
```

```
[1] 0
```

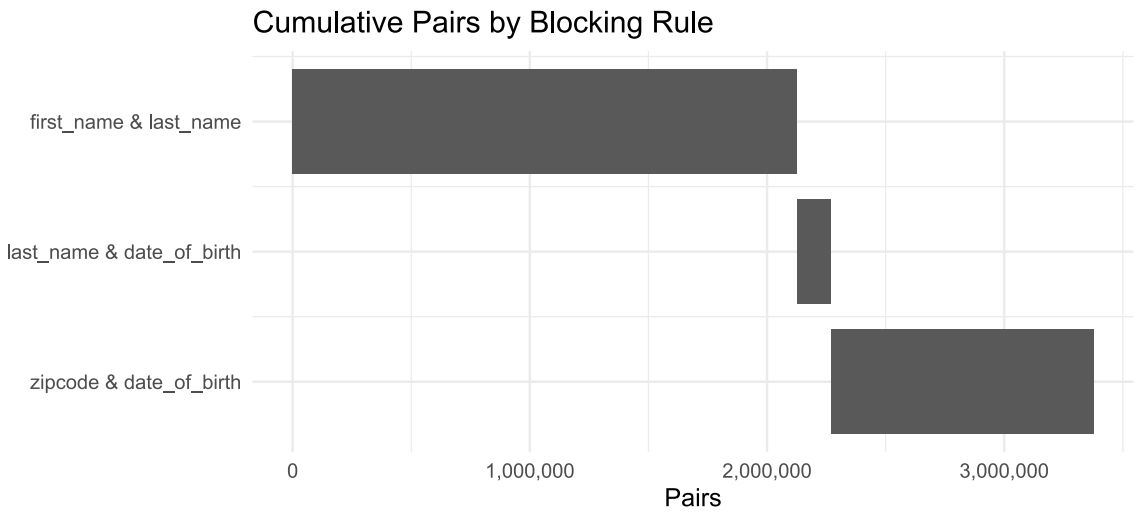
```
zero_high_left <- tbl(zero_high_con, 'zero_high_left')
zero_high_right <- tbl(zero_high_con, 'zero_high_right')
```

Blocking

```
tic.clearlog()
tic('zero_high_blocking', quiet = TRUE)
zero_high_blocking <- il_count_pairs(
  zero_high_left,
  zero_high_right,
  block_on(first_name, last_name),
  block_on(last_name, date_of_birth),
  block_on(zipcode, date_of_birth),
  con = zero_high_con,
  link_type = 'link'
)
zero_high_blocking_timing <- toc(log = TRUE, quiet = TRUE)
zero_high_blocking_time <- as.numeric(
  zero_high_blocking_timing$toc - zero_high_blocking_timing$tic
)
zero_high_candidate_pairs <- max(zero_high_blocking$cumulative_pairs)
zero_high_cartesian_share <- max(zero_high_blocking$pct_of_cartesian)
```

This links the clean census extract to the high-noise administrative extract. The candidate universe is 3,376,207 pairs, or 0.03% of the full cross-product.

```
autoplot(zero_high_blocking)
```



Model fit

```
tic.clearlog()
tic('zero_high_fit', quiet = TRUE)
zero_high_model <- il_model(
  zero_high_left,
```

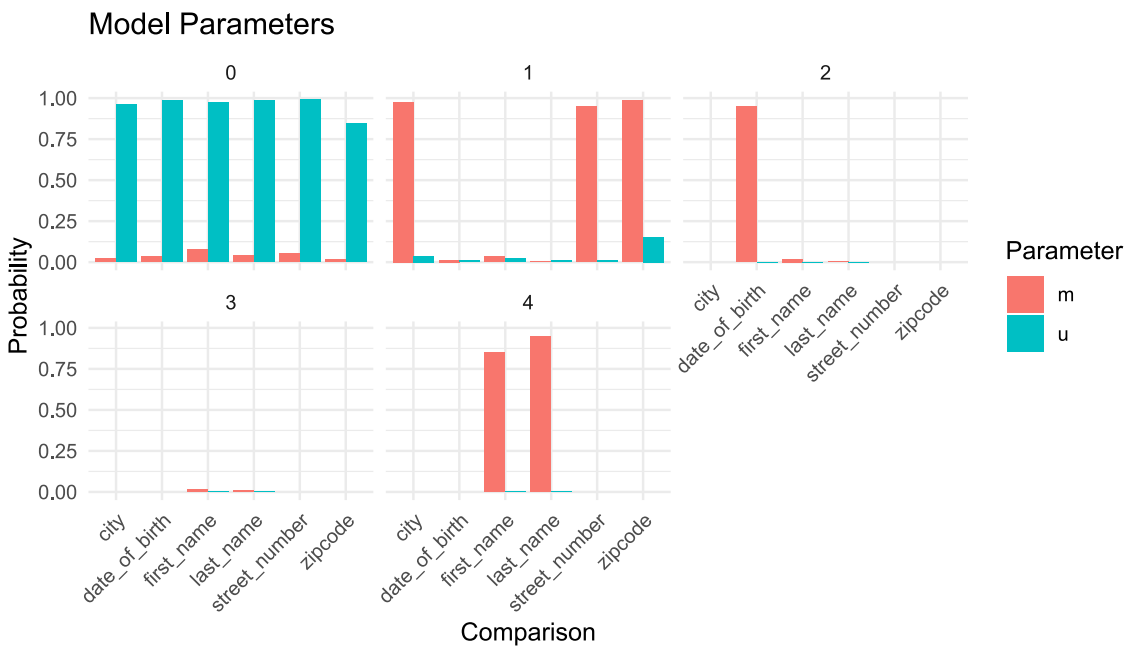
```
  zero_high_right,
  spec = common_spec,
  con = zero_high_con,
  link_type = 'link'
) |>
  il_estimate_prior(
    block_on(first_name, last_name, date_of_birth),
    recall = 0.8
  ) |>
  il_estimate_u(max_pairs = 5e5) |>
  il_estimate_em(
    blocking = block_on(first_name, last_name),
    max_iterations = 8L
  ) |>
  il_estimate_em(
    blocking = block_on(zipcode, date_of_birth),
    max_iterations = 8L
  )
```

```
EM trained: date_of_birth, street_number, city, and zipcode | skipped (blocked
on): first_name and last_name
EM trained: first_name, last_name, street_number, and city | skipped (blocked
on): date_of_birth and zipcode
```

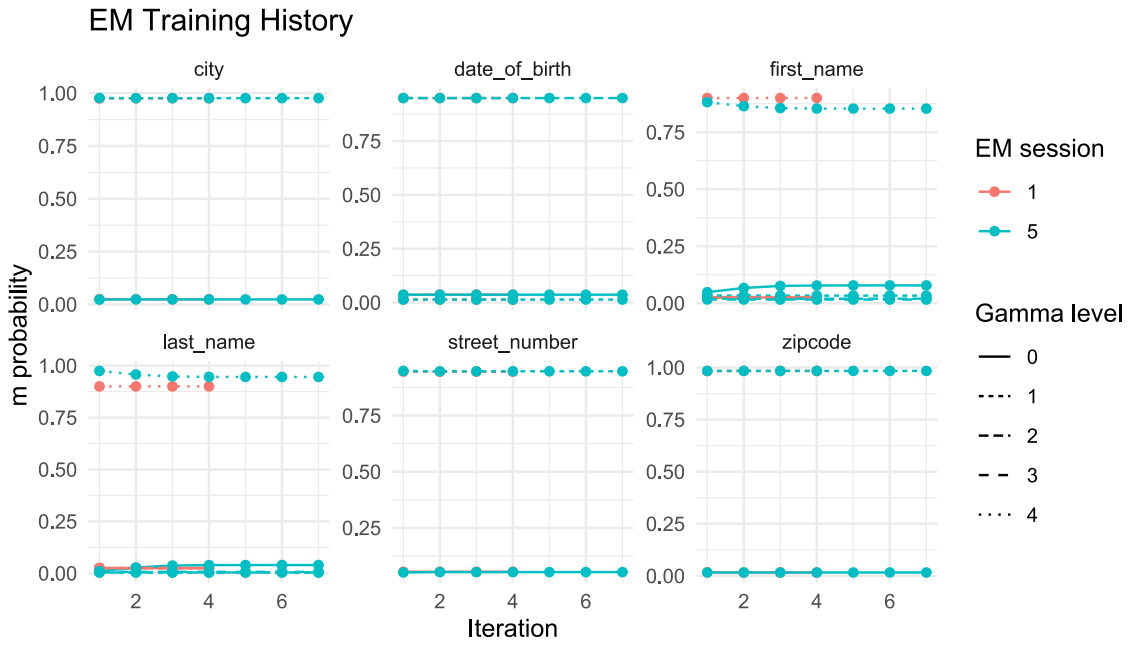
```
zero_high_fit_timing <- toc(log = TRUE, quiet = TRUE)
zero_high_fit_time <- as.numeric(
  zero_high_fit_timing$toc - zero_high_fit_timing$tic
)
```

The fit step takes 7.6 seconds. This is the hardest one-sided linkage task in the document, so the fitted parameter separation is weaker than in the no-noise-to-default case.

```
autoplot(zero_high_model, type = 'parameters')
```




```
il_training_history(zero_high_model) |>
  autoplot()
```

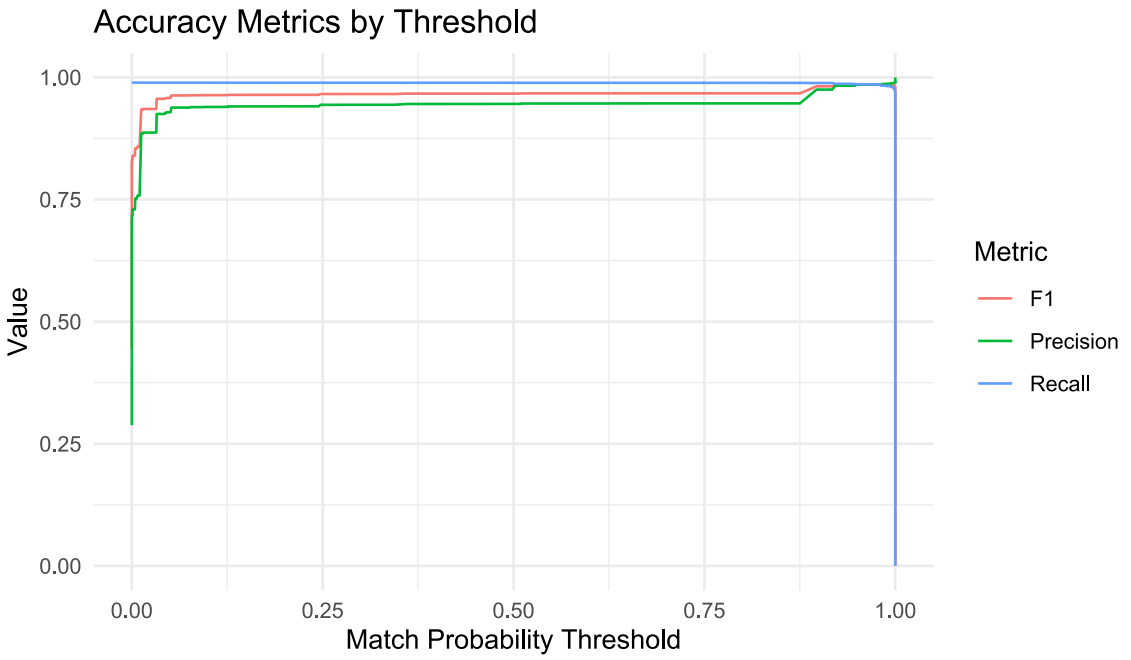


Accuracy

```
tic.clearlog()
tic('zero_high_accuracy', quiet = TRUE)
zero_high_accuracy <- il_accuracy(
  zero_high_model,
  labels_col = 'simulant_id'
)
zero_high_accuracy_timing <- toc(log = TRUE, quiet = TRUE)
zero_high_accuracy_time <- as.numeric(
  zero_high_accuracy_timing$toc - zero_high_accuracy_timing$tic
)
zero_high_best <- zero_high_accuracy |>
  slice_max(f1, n = 1, with_ties = FALSE)
```

At the best threshold (0.948), precision is 98.49%, recall is 98.57%, and F1 is 0.985. Accuracy evaluation takes 61.5 seconds.

```
autoplot(zero_high_accuracy)
```



Prediction

```
tic.clearlog()
tic('zero_high_prediction', quiet = TRUE)
zero_high_pairs <- predict(
  zero_high_model,
```

```
  threshold = zero_high_best$threshold
)
zero_high_prediction_timing <- toc(log = TRUE, quiet = TRUE)
zero_high_prediction_time <- as.numeric(
  zero_high_prediction_timing$toc - zero_high_prediction_timing$tic
)
zero_high_predicted_links <- nrow(zero_high_pairs)

tic.clearlog()
tic('zero_high_prediction_correctness', quiet = TRUE)
zero_high_confusion <- il_confusion_matrix(
  zero_high_model,
  threshold = zero_high_best$threshold,
  labels_col = 'simulant_id'
)
zero_high_prediction_correctness_timing <- toc(log = TRUE, quiet = TRUE)
zero_high_prediction_correctness_time <- as.numeric(
  zero_high_prediction_correctness_timing$toc -
    zero_high_prediction_correctness_timing$tic
)
```

At the best F1 threshold, `predict()` returns 984,008 links in 2.3 seconds. Thresholded prediction correctness is precision 98.49%, recall 98.57%, and F1 0.985.

Timing

```
zero_high_timings <- tibble::tibble(
  stage = c(
    'Blocking analysis',
    'Model fitting',
    'Accuracy evaluation',
    'Prediction',
    'Prediction correctness',
    'Total'
  ),
  seconds = c(
    zero_high_blocking_time,
    zero_high_fit_time,
    zero_high_accuracy_time,
    zero_high_prediction_time,
    zero_high_prediction_correctness_time,
    zero_high_blocking_time +
      zero_high_fit_time +
      zero_high_accuracy_time +
      zero_high_prediction_time +
      zero_high_prediction_correctness_time
  )
)
```

```
zero_high_timings
```

```
# A tibble: 6 × 2
  stage                  seconds
  <chr>                    <dbl>
1 Blocking analysis         2.98
2 Model fitting             7.61
3 Accuracy evaluation      61.5
4 Prediction                2.34
5 Prediction correctness   14.2
6 Total                    88.7
```

Default noise to high

```
default_high_con <- DBI::dbConnect(duckdb::duckdb())

DBI::dbExecute(
  default_high_con,
  paste0(
    'CREATE VIEW default_high_left AS ',
    "SELECT * FROM read_parquet('",
    default_path,
    "')"
  )
)
```

```
[1] 0
```

```
DBI::dbExecute(
  default_high_con,
  paste0(
    'CREATE VIEW default_high_right AS ',
    "SELECT * FROM read_parquet('",
    high_path,
    "')"
  )
)
```

```
[1] 0
```

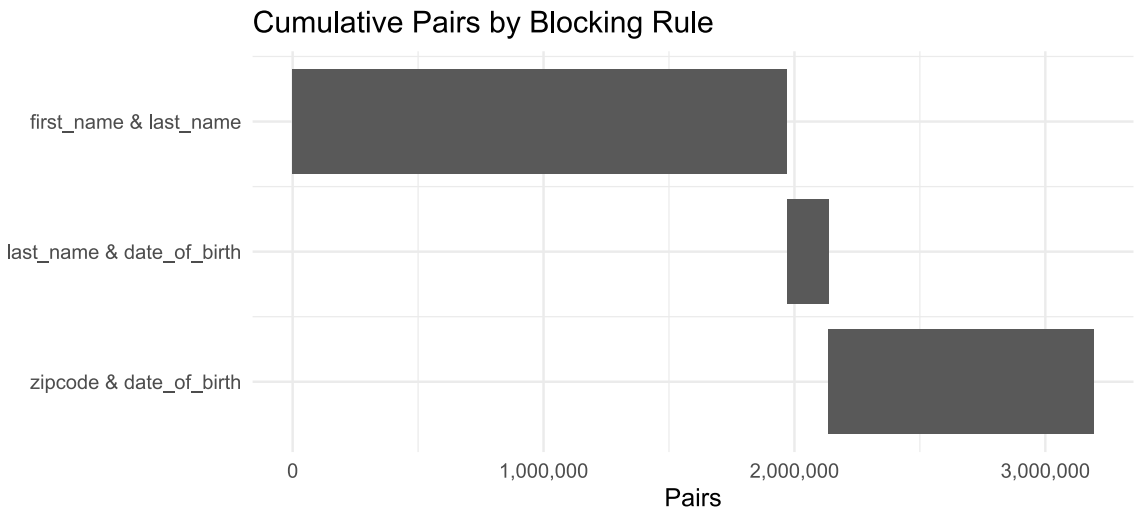
```
default_high_left <- tbl(default_high_con, 'default_high_left')
default_high_right <- tbl(default_high_con, 'default_high_right')
```

Blocking

```
tic.clearlog()
tic('default_high_blocking', quiet = TRUE)
default_high_blocking <- il_count_pairs(
  default_high_left,
  default_high_right,
  block_on(first_name, last_name),
  block_on(last_name, date_of_birth),
  block_on(zipcode, date_of_birth),
  con = default_high_con,
  link_type = 'link'
)
default_high_blocking_timing <- toc(log = TRUE, quiet = TRUE)
default_high_blocking_time <- as.numeric(
  default_high_blocking_timing$toc - default_high_blocking_timing$tic
)
default_high_candidate_pairs <- max(default_high_blocking$cumulative_pairs)
default_high_cartesian_share <- max(default_high_blocking$pct_of_cartesian)
```

This is the realistic administrative linkage case, with noise on both sides. The candidate universe is 3,191,749 pairs, or 0.03% of the full cross-product.

```
autoplot(default_high_blocking)
```



Model fit

```
tic.clearlog()
tic('default_high_fit', quiet = TRUE)
default_high_model <- il_model(
  default_high_left,
```

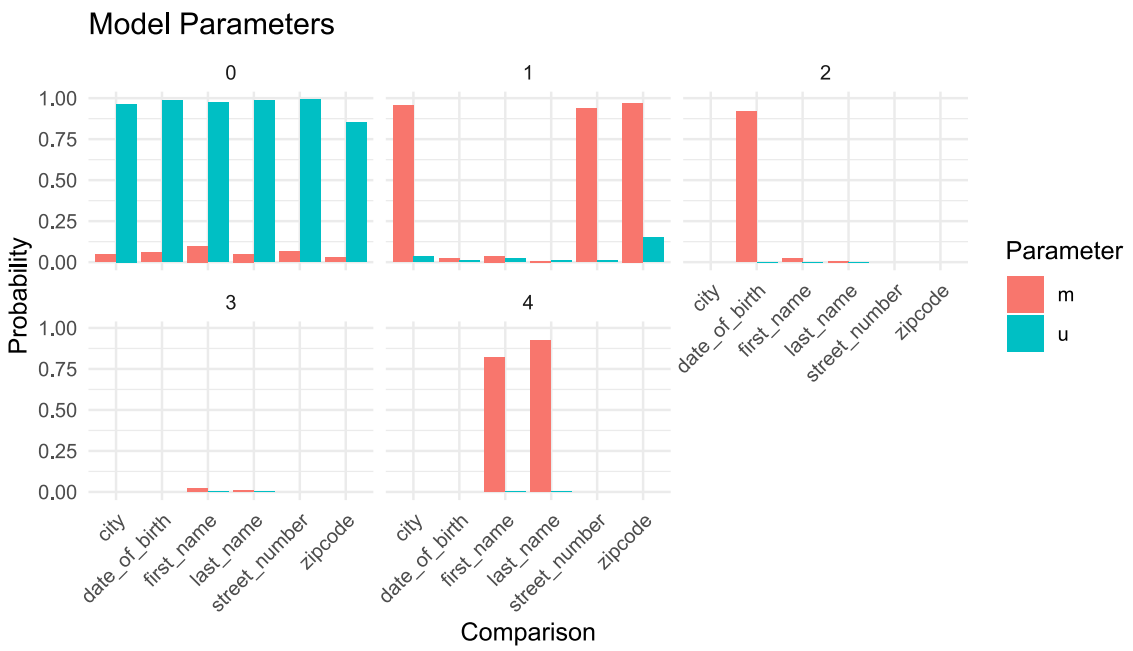
```
  default_high_right,
  spec = common_spec,
  con = default_high_con,
  link_type = 'link'
) |>
  il_estimate_prior(
    block_on(first_name, last_name, date_of_birth),
    recall = 0.8
  ) |>
  il_estimate_u(max_pairs = 5e5) |>
  il_estimate_em(
    blocking = block_on(first_name, last_name),
    max_iterations = 8L
  ) |>
  il_estimate_em(
    blocking = block_on(zipcode, date_of_birth),
    max_iterations = 8L
  )
```

```
EM trained: date_of_birth, street_number, city, and zipcode | skipped (blocked
on): first_name and last_name
EM trained: first_name, last_name, street_number, and city | skipped (blocked
on): date_of_birth and zipcode
```

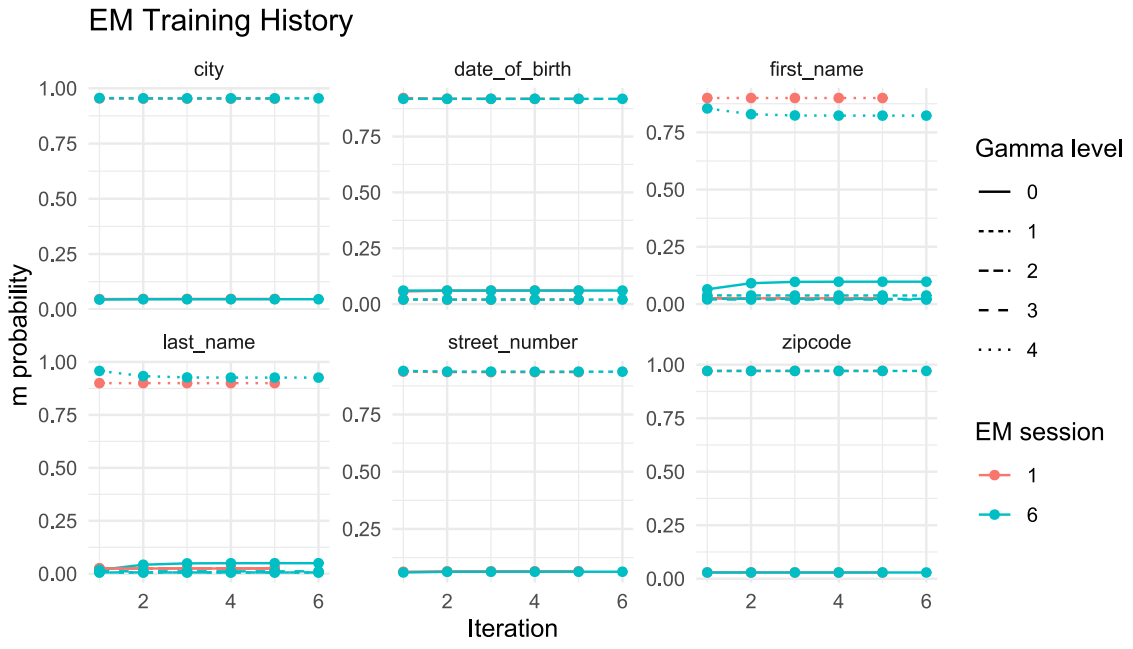
```
default_high_fit_timing <- toc(log = TRUE, quiet = TRUE)
default_high_fit_time <- as.numeric(
  default_high_fit_timing$toc - default_high_fit_timing$tic
)
```

The fit step takes 7.2 seconds. Because both sides are noisy, this is the most realistic two-file linkage benchmark in the set.

```
autoplot(default_high_model, type = 'parameters')
```




```
il_training_history(default_high_model) |>
  autoplot()
```

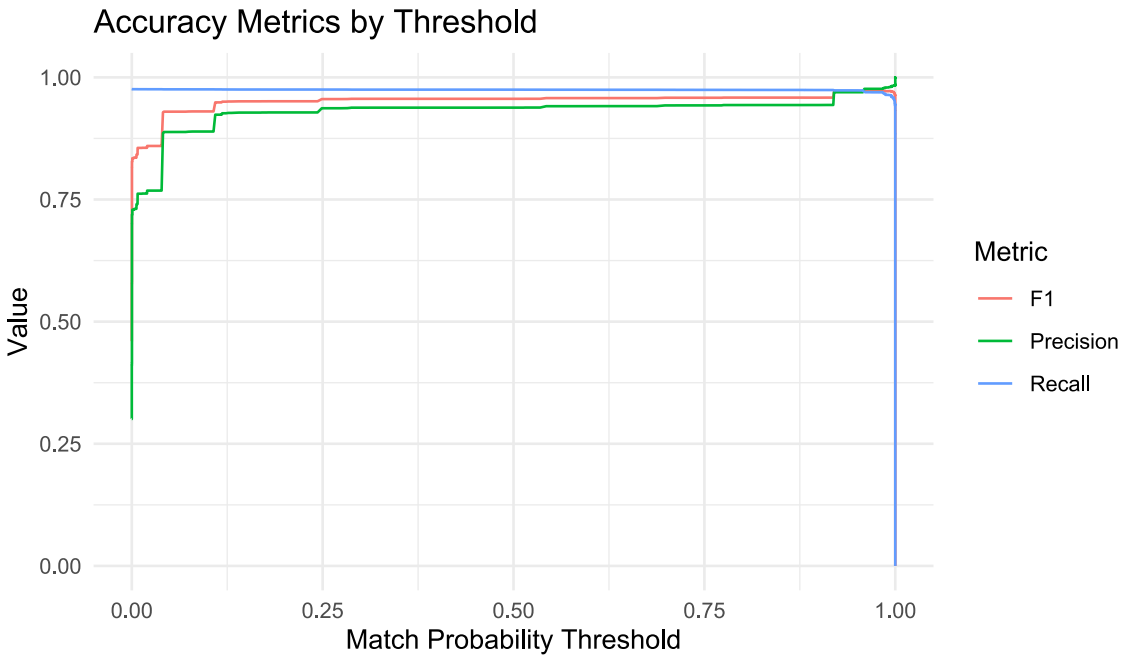


Accuracy

```
tic.clearlog()
tic('default_high_accuracy', quiet = TRUE)
default_high_accuracy <- il_accuracy(
  default_high_model,
  labels_col = 'simulant_id'
)
default_high_accuracy_timing <- toc(log = TRUE, quiet = TRUE)
default_high_accuracy_time <- as.numeric(
  default_high_accuracy_timing$toc - default_high_accuracy_timing$tic
)
default_high_best <- default_high_accuracy |>
  slice_max(f1, n = 1, with_ties = FALSE)
```

At the best threshold (0.959), precision is 97.64%, recall is 97.05%, and F1 is 0.973. Accuracy evaluation takes 70.3 seconds.

```
autoplot(default_high_accuracy)
```



Prediction

```
tic.clearlog()
tic('default_high_prediction', quiet = TRUE)
default_high_pairs <- predict(
  default_high_model,
```

```
  threshold = default_high_best$threshold
)
default_high_prediction_timing <- toc(log = TRUE, quiet = TRUE)
default_high_prediction_time <- as.numeric(
  default_high_prediction_timing$toc - default_high_prediction_timing$tic
)
default_high_predicted_links <- nrow(default_high_pairs)

tic.clearlog()
tic('default_high_prediction_correctness', quiet = TRUE)
default_high_confusion <- il_confusion_matrix(
  default_high_model,
  threshold = default_high_best$threshold,
  labels_col = 'simulant_id'
)
default_high_prediction_correctness_timing <- toc(log = TRUE, quiet = TRUE)
default_high_prediction_correctness_time <- as.numeric(
  default_high_prediction_correctness_timing$toc -
    default_high_prediction_correctness_timing$tic
)
```

At the best F1 threshold, `predict()` returns 976,238 links in 2.4 seconds. Thresholded prediction correctness is precision 97.64%, recall 97.05%, and F1 0.973.

Timing

```
default_high_timings <- tibble::tibble(
  stage = c(
    'Blocking analysis',
    'Model fitting',
    'Accuracy evaluation',
    'Prediction',
    'Prediction correctness',
    'Total'
  ),
  seconds = c(
    default_high_blocking_time,
    default_high_fit_time,
    default_high_accuracy_time,
    default_high_prediction_time,
    default_high_prediction_correctness_time,
    default_high_blocking_time +
      default_high_fit_time +
      default_high_accuracy_time +
      default_high_prediction_time +
      default_high_prediction_correctness_time
  )
)
```

```
default_high_timings
```

```
# A tibble: 6 × 2
  stage                  seconds
  <chr>                    <dbl>
1 Blocking analysis         2.48
2 Model fitting             7.24
3 Accuracy evaluation      70.3
4 Prediction                2.35
5 Prediction correctness   13.3
6 Total                    95.7
```

Stacked zero, default, and high noise

```
stacked_dedupe_con <- DBI::dbConnect(duckdb::duckdb())

DBI::dbExecute(
  stacked_dedupe_con,
  paste0(
    'CREATE VIEW stacked_dedupe_input AS ',
    "SELECT 'zero' AS source_file, * FROM read_parquet('",
    zero_path,
    "') ",
    'UNION ALL ',
    "SELECT 'default' AS source_file, * FROM read_parquet('",
    default_path,
    "') ",
    'UNION ALL ',
    "SELECT 'high' AS source_file, * FROM read_parquet('",
    high_path,
    "')"
  )
)
```

```
[1] 0
```

```
stacked_dedupe_input <- tbl(stacked_dedupe_con, 'stacked_dedupe_input')
```

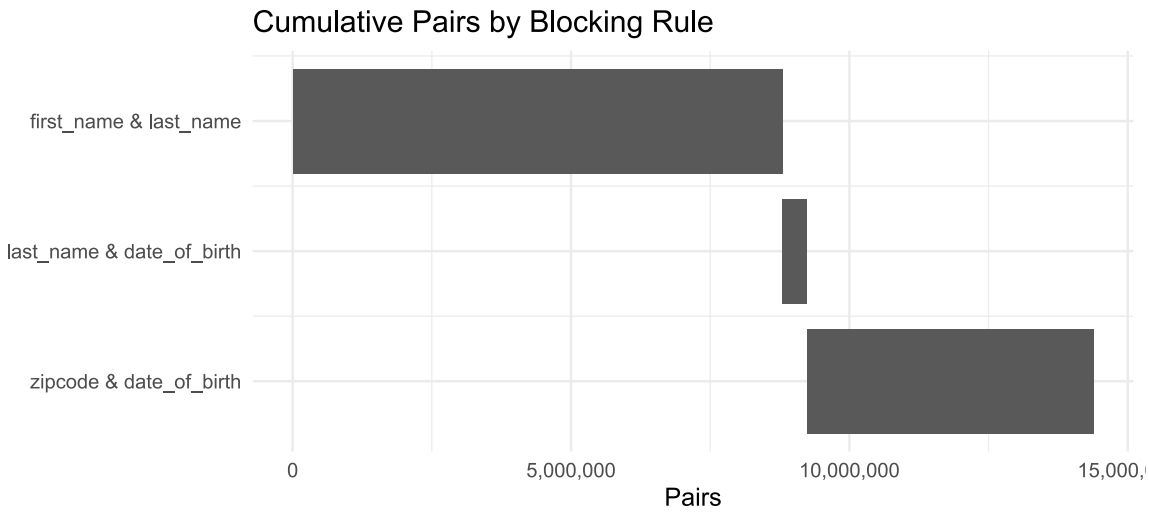
Blocking

```
tic.clearlog()
tic('stacked_dedupe_blocking', quiet = TRUE)
stacked_dedupe_blocking <- il_count_pairs(
```

```
  stacked_dedupe_input,
  block_on(first_name, last_name),
  block_on(last_name, date_of_birth),
  block_on(zipcode, date_of_birth),
  con = stacked_dedupe_con,
  link_type = 'dedupe'
)
stacked_dedupe_blocking_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_blocking_time <- as.numeric(
  stacked_dedupe_blocking_timing$toc - stacked_dedupe_blocking_timing$tic
)
stacked_dedupe_candidate_pairs <-
max(stacked_dedupe_blocking$cumulative_pairs)
stacked_dedupe_cartesian_share <-
max(stacked_dedupe_blocking$pct_of_cartesian)
```

This stacks the clean, default-noise, and high-noise census extracts and deduplicates the combined table. The blocking rules cut the search to 14,383,439 candidate pairs, or 0.03% of all within-table pairs.

```
autoplot(stacked_dedupe_blocking)
```



Model fit

```
tic.clearlog()
tic('stacked_dedupe_fit', quiet = TRUE)
stacked_dedupe_model <- il_model(
  stacked_dedupe_input,
  spec = common_spec,
  con = stacked_dedupe_con,
```

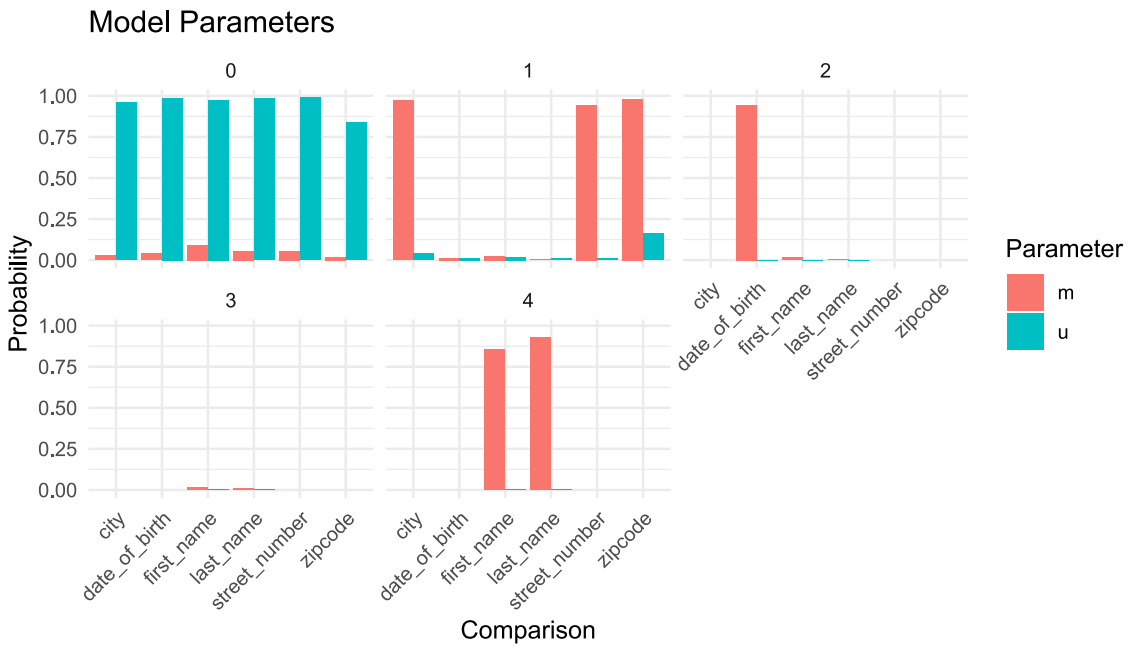
```
  link_type = 'dedupe'
) |>
  il_estimate_prior(
    block_on(first_name, last_name, date_of_birth),
    recall = 0.8
  ) |>
  il_estimate_u(max_pairs = 5e5) |>
  il_estimate_em(
    blocking = block_on(first_name, last_name),
    max_iterations = 8L
  ) |>
  il_estimate_em(
    blocking = block_on(zipcode, date_of_birth),
    max_iterations = 8L
  )
```

```
EM trained: date_of_birth, street_number, city, and zipcode | skipped (blocked
on): first_name and last_name
EM trained: first_name, last_name, street_number, and city | skipped (blocked
on): date_of_birth and zipcode
```

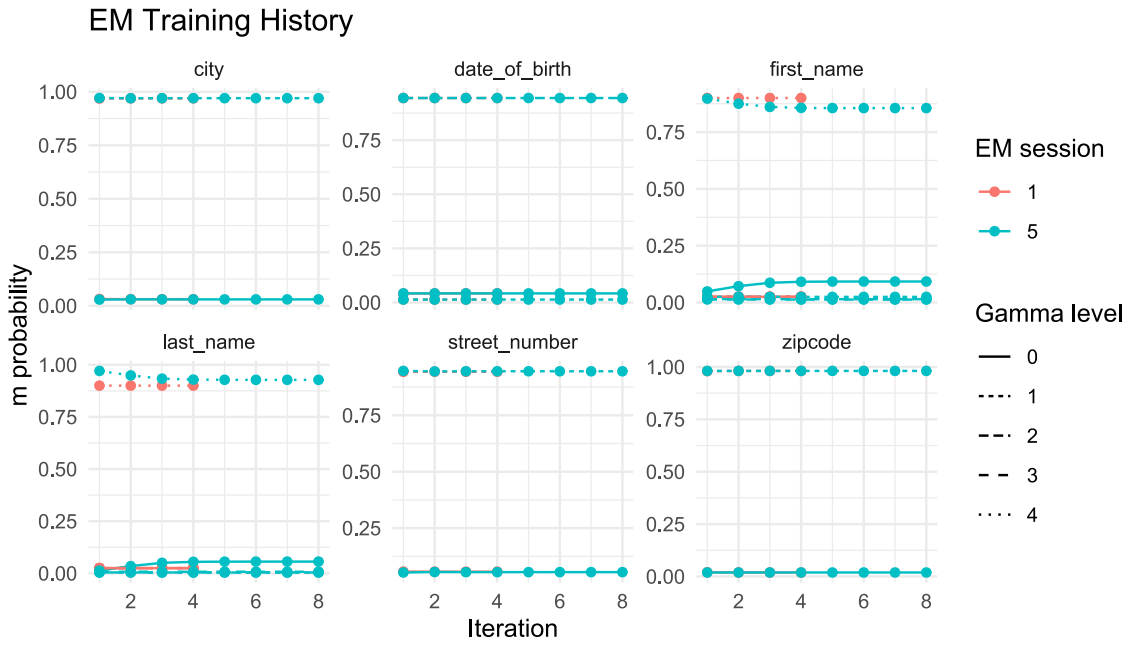
```
stacked_dedupe_fit_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_fit_time <- as.numeric(
  stacked_dedupe_fit_timing$toc - stacked_dedupe_fit_timing$tic
)
```

This fit step takes 15.9 seconds. The stacked deduplication case is the heaviest benchmark because it includes cross-source duplicates and the small number of within-source duplicates present in the noisy files.

```
autoplot(stacked_dedupe_model, type = 'parameters')
```




```
il_training_history(stacked_dedupe_model) |>
  autoplot()
```

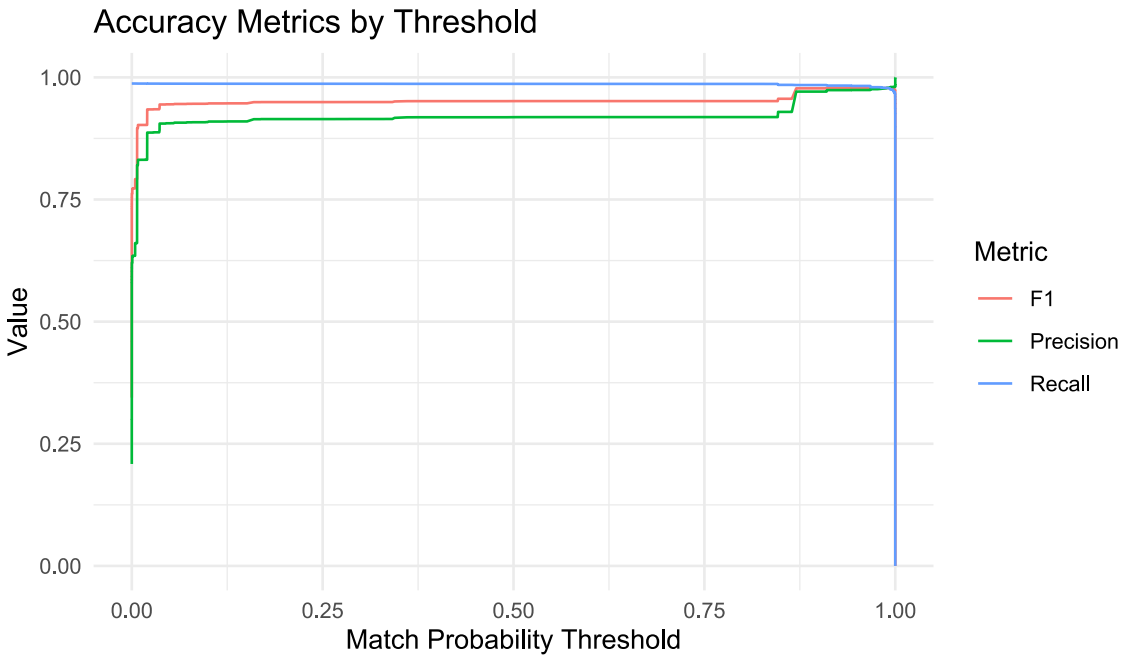


Accuracy

```
tic.clearlog()
tic('stacked_dedupe_accuracy', quiet = TRUE)
stacked_dedupe_accuracy <- il_accuracy(
  stacked_dedupe_model,
  labels_col = 'simulant_id'
)
stacked_dedupe_accuracy_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_accuracy_time <- as.numeric(
  stacked_dedupe_accuracy_timing$toc - stacked_dedupe_accuracy_timing$tic
)
stacked_dedupe_best <- stacked_dedupe_accuracy |>
  slice_max(f1, n = 1, with_ties = FALSE)
```

At the best threshold (0.993), precision is 97.93%, recall is 97.78%, and F1 is 0.979. Accuracy evaluation takes 312.6 seconds.

```
autoplot(stacked_dedupe_accuracy)
```



Prediction

```
tic.clearlog()
tic('stacked_dedupe_prediction', quiet = TRUE)
stacked_dedupe_pairs <- predict(
  stacked_dedupe_model,
```

```
  threshold = stacked_dedupe_best$threshold
)
stacked_dedupe_prediction_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_prediction_time <- as.numeric(
  stacked_dedupe_prediction_timing$toc - stacked_dedupe_prediction_timing$tic
)
stacked_dedupe_predicted_pairs <- nrow(stacked_dedupe_pairs)

tic.clearlog()
tic('stacked_dedupe_prediction_correctness', quiet = TRUE)
stacked_dedupe_confusion <- il_confusion_matrix(
  stacked_dedupe_model,
  threshold = stacked_dedupe_best$threshold,
  labels_col = 'simulant_id'
)
stacked_dedupe_prediction_correctness_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_prediction_correctness_time <- as.numeric(
  stacked_dedupe_prediction_correctness_timing$toc -
    stacked_dedupe_prediction_correctness_timing$tic
)
```

At the best F1 threshold, `predict()` returns 3,029,788 scored duplicate pairs in 8.5 seconds. Pairwise correctness at that threshold is precision 97.93%, recall 97.78%, and F1 0.979.

Clustering

```
tic.clearlog()
tic('stacked_dedupe_clustering', quiet = TRUE)
stacked_dedupe_clusters <- il_cluster(stacked_dedupe_pairs)
stacked_dedupe_clustering_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_clustering_time <- as.numeric(
  stacked_dedupe_clustering_timing$toc - stacked_dedupe_clustering_timing$tic
)
stacked_dedupe_cluster_sizes <- stacked_dedupe_clusters |>
  count(cluster_id, name = 'cluster_size', sort = TRUE)
stacked_dedupe_n_clusters <- nrow(stacked_dedupe_cluster_sizes)
stacked_dedupe_largest_cluster <- if (nrow(stacked_dedupe_cluster_sizes) > 0)
{
  max(stacked_dedupe_cluster_sizes$cluster_size)
} else {
  0L
}

tic.clearlog()
tic('stacked_dedupe_cluster_correctness', quiet = TRUE)
stacked_dedupe_cluster_confusion <- il_cluster_confusion_matrix(
  stacked_dedupe_model,
  labels_col = 'simulant_id',
```

```
  threshold = stacked_dedupe_best$threshold
)
stacked_dedupe_cluster_correctness_timing <- toc(log = TRUE, quiet = TRUE)
stacked_dedupe_cluster_correctness_time <- as.numeric(
  stacked_dedupe_cluster_correctness_timing$toc -
    stacked_dedupe_cluster_correctness_timing$tic
)
```

Connected-components clustering yields 1,055,866 clusters, with largest cluster size 14, in 24.8 seconds. Cluster-level duplicate detection correctness is precision 99.18%, recall 98.66%, and F1 0.989.

Timing

```
stacked_dedupe_timings <- tibble::tibble(
  stage = c(
    'Blocking analysis',
    'Model fitting',
    'Accuracy evaluation',
    'Prediction',
    'Prediction correctness',
    'Clustering',
    'Cluster correctness',
    'Total'
  ),
  seconds = c(
    stacked_dedupe_blocking_time,
    stacked_dedupe_fit_time,
    stacked_dedupe_accuracy_time,
    stacked_dedupe_prediction_time,
    stacked_dedupe_prediction_correctness_time,
    stacked_dedupe_clustering_time,
    stacked_dedupe_cluster_correctness_time,
    stacked_dedupe_blocking_time +
      stacked_dedupe_fit_time +
      stacked_dedupe_accuracy_time +
      stacked_dedupe_prediction_time +
      stacked_dedupe_prediction_correctness_time +
      stacked_dedupe_clustering_time +
      stacked_dedupe_cluster_correctness_time
  )
)

stacked_dedupe_timings
```

```
# A tibble: 8 × 2
  stage                  seconds
```

```
  <chr>                    <dbl>
1 Blocking analysis         5.19
2 Model fitting            15.9
3 Accuracy evaluation     313.
4 Prediction                8.5
5 Prediction correctness   53.1
6 Clustering               24.8
7 Cluster correctness      11.7
8 Total                   432.
```